

MemSnap: Automated Linux Memory Forensics with AI-Driven Threat Detection

Technical Report - Proof of Concept

Authors

Karim Elkhanoufi,
Karim Amaayou,
Moataz Bilah Aksaim

Supervisor

Prof. Manale Boughanja

CONTENTS

I.	Introduction	1
II.	System Architecture	2
III.	Infrastructure	3
	1. Lightweight Virtual Machine	3
	2. Memory Acquisition	3
	3. Volatility 3 and Symbol Table	3
IV.	Malware Components	4
	1. Rootkit	4
	2. Code Injection	4
	3. Backdoor	4
V.	Feature Extraction	5
VI.	AI Model	6
	1. Dataset	6
	2. Classifier	6
VII.	Showcase	7
	1. Infection & Capture	7
	2. Feature Extraction	7
	3. Model Training	8
	4. CLI Detection	8
	5. Web Interface	9
VIII.	Conclusion	11
IX.	Future Work	12

FIGURES

Figure 1	MemSnap pipeline overview	2
Figure 2	Repository Structure	2
Figure 3	infection and capture dump	7
Figure 4	Volatility plugin execution and CSV output	7
Figure 5	Extracted features	8
Figure 6	LOO-CV results and model serialisation	8
Figure 7	Single-dump verdict	8
Figure 8	Upload the dump and model	9
Figure 9	Extraction features	9
Figure 10	Threat verdict	10

TABLES

Table 1	Extracted forensic features and their relevance	5
Table 2	Training dataset	6
Table 3	Model's accuracy	6

I. INTRODUCTION

Memsnap - Memory forensics is an essential technique for detecting advanced malware that operates entirely in RAM, bypassing disk-based defenses. However, manual analysis of memory dumps is time-consuming and requires deep expertise. **MemSnap** automates this process by:

1. Infecting a controlled Linux vm with malware components (rootkit, code injection, backdoor).
2. Capturing a compressed memory dump with ***AVML***.
3. Extracting forensic features using ***Volatility 3***.
4. Training a machine learning classifier to detect three threat categories.
5. Providing a **CLI** and a **web** interface for single-dump verdicts.

This report describes the design, implementation, and evaluation of the MemSnap proof-of-concept pipeline.

II. SYSTEM ARCHITECTURE



Figure 1: MemSnap pipeline overview

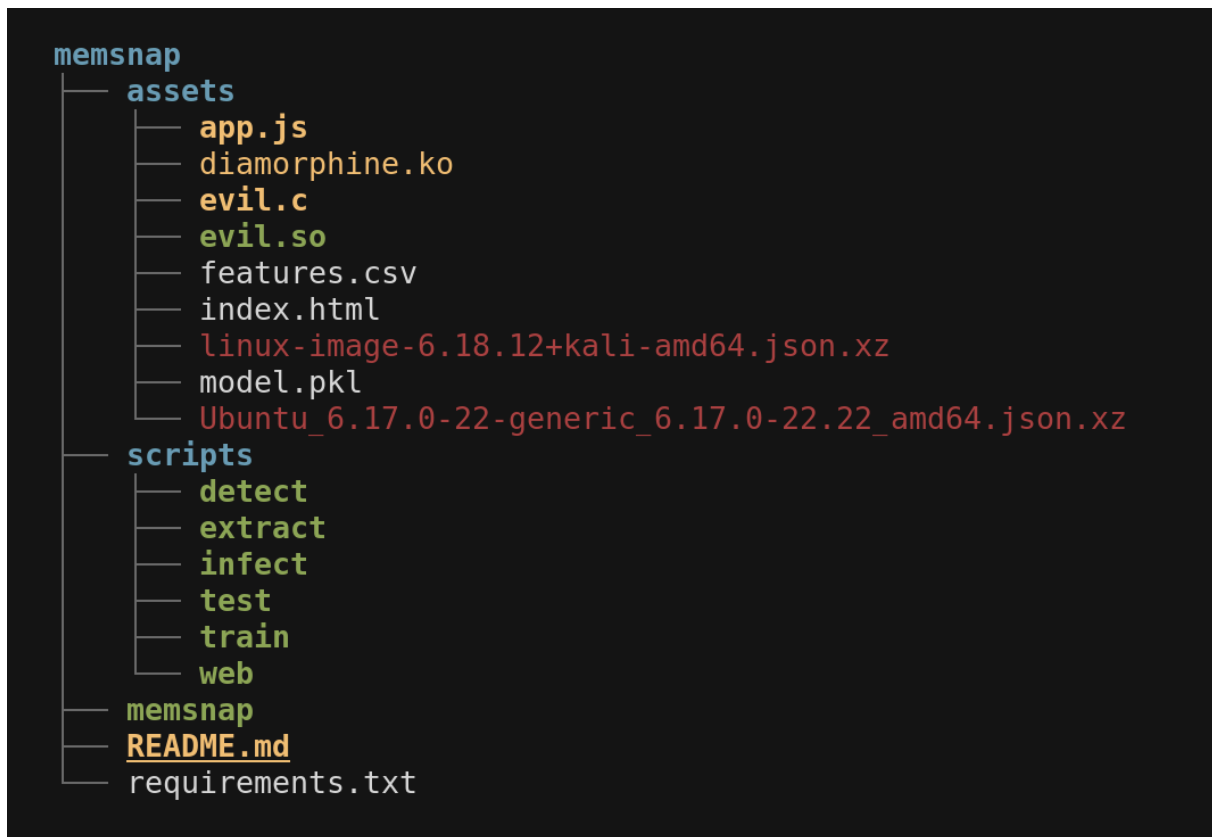


Figure 2: Repository Structure

III. INFRASTRUCTURE

1. Lightweight Virtual Machine

A minimal Ubuntu 24.04 VM was prepared for the experiments.

Unnecessary services, packages were removed, leaving a headless system that idles at approximately **159 MB** of RAM.

The hypervisor is configured with **512 MB** of memory, leaving ample room for malware execution and memory capture.

- **Kernel version:** 6.17.0-22-generic

2. Memory Acquisition

[AVML](#) is used to capture memory. It compresses dump files on-the-fly, reducing a **1 GB** raw dump to approximately **180-200 MB**.

AVML is automatically downloaded into `$HOME/.local/bin/` on first use.

3. Volatility 3 and Symbol Table

The [Volatility 3](#) framework (v2.28.0) is installed inside a Python virtual environment.

A pre-generated ISF symbol table for kernel 6.17.0-22-generic is placed in `assets/`, enabling all plugins to function without additional setup.

Build ISF symbol table for your specific kernel version by following [this](#) tutorial.

IV. MALWARE COMPONENTS

Three distinct threat types are used to train the AI model. Each leaves a characteristic memory artifact that can be extracted by Volatility.

1. Rootkit

The kernel module *Diamorphine* hides itself from module listings, hides processes from `/proc`.

After loading, the module is invisible to `lsmod`, but Volatility's `linux.hidden_modules` plugin can detect it by comparing the kernel's module list with the list of loaded modules.

2. Code Injection

A malicious shared library (`evil.so`) is injected into a `sleep 9999` process using the `LD_PRELOAD` environment variable.

To make this injection detectable in memory dumps, a unique marker file `/tmp/injection_marker` is created and kept open by a dedicated `tail` process.

Volatility's `linux.lsof.Lsof` plugin reads the kernel's `fdtable` chain directly from physical memory, so it can see the open file descriptor even if the rootkit hides the process from `/proc`.

3. Backdoor

The `ncat` utility is installed on the VM, and a listening TCP socket is bound to port `4444` with a shell attached. This opens a backdoor that an attacker could use to gain remote access. The `linux.sockstat` plugin reveals all open sockets;

a filter for `Source Port == 4444` and `State == LISTEN` identifies the backdoor.

V. FEATURE EXTRACTION

For each memory dump, the following Volatility 3 plugins are executed:

- `linux.psscan`
- `linux.check_syscall`
- `linux.hidden_modules`
- `linux.sockstat`
- `linux.lsof.Lsof`

From their outputs, a set of numeric features is derived:

Feature	Description	Relevance
<code>hidden_module_count</code>	Number of hidden kernel modules detected	Perfectly separates rootkit dumps (value = 1) from clean ones (value = 0)
<code>has_injection</code>	1 if any process has an open file descriptor containing the string <code>injection_marker</code> , else 0	Perfectly separates injection dumps
<code>high_port_listeners</code>	Count of sockets listening on port 4444	Identifies backdoor dumps; however, one non-backdoor dump also shows this feature (see Section 7.1)
<code>total_sockets</code>	Total open sockets	Supplementary information
<code>hidden_proc_count</code>	Difference between PIDs seen by <code>psscan</code> and <code>pslist</code>	Potentially indicative of hidden processes (noisy in practice)
<code>hooked_syscall_count</code>	Number of hooked syscalls (from <code>check_syscall</code>)	Always zero on this kernel; retained for completeness

Table 1: Extracted forensic features and their relevance

The features are written to `features.csv`, which serves as the training dataset.

VI. AI MODEL

1. Dataset

A balanced dataset of 8 memory dumps was generated, covering every combination of the three threat categories:

Dump Name	Rootkit	Injection	Backdoor
memdump_..._clean.raw	0	0	0
memdump_..._rootkit.raw	1	0	0
memdump_..._injection.raw	0	1	0
memdump_..._backdoor.raw	0	0	1
memdump_..._rootkit_injection.raw	1	1	0
memdump_..._injection_backdoor.raw	0	1	1
memdump_..._rootkit_backdoor.raw	1	0	1
memdump_..._rootkit_injection_backdoor.raw	1	1	1

Table 2: Training dataset

The small size is sufficient for a proof-of-concept because the chosen features provide strong, nearly perfect separation for two of the three classes.

2. Classifier

A *multi-output Decision Tree* classifier was chosen for its simplicity and interpretability. Using *leave-one-out cross-validation*, the model achieves:

	Rootkit	Injection	Backdoor
Accuracy	1.00	1.00	0.38

Table 3: Model's accuracy

VII. SHOWCASE

1. Infection & Capture

Deploys the selected malware component and optionally triggers an AVML memory capture. The `-c` flag captures a dump immediately after infection.

```
memsnap infect -a ~/shared -i rootkit,injection,backdoor
```

```

ubuntu@server:~/memsnap$
ubuntu@server:~/memsnap$
ubuntu@server:~/memsnap$
ubuntu@server:~/memsnap$ memsnap infect -a ~/shared -i rootkit,injection,backdoor
[*] Rootkit appears to be already loaded (detected via /sys).
[+] Code injected (LD_PRELOAD) into PID 1634
[+] Marker sentinel PID 1636 holding /tmp/injection_marker open
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ncat is already the newest version (7.94+git20230807.3be01efb1+dfsg-3build2).
0 upgraded, 0 newly installed, 0 to remove and 1 not upgraded.
[+] Bind shell on port 4444
[+] Infection complete.
[*] Using system-installed avml: /home/ubuntu/.local/bin/avml
[*] Capturing memory to /home/ubuntu/shared/memdump_20260511_153555_rootkit_injection_backdoor.raw ...
[+] Memory dump saved: /home/ubuntu/shared/memdump_20260511_153555_rootkit_injection_backdoor.raw
ubuntu@server:~/memsnap$
  
```

Annotations in the image:

- A red box around `~/shared` is labeled "output directory (dump)".
- A red box around `rootkit,injection,backdoor` is labeled "<- list of components infect the machine with".

Figure 3: infection and capture dump

2. Feature Extraction

Runs all Volatility 3 plugins against every dump in a directory and writes the resulting numeric features to `features.csv`.

```
memsnap extract ~/shared ~/demo
```

```

(venv) [karim@T460 forensic-lab]$
(venv) [karim@T460 forensic-lab]$
(venv) [karim@T460 forensic-lab]$
(venv) [karim@T460 forensic-lab]$ memsnap extract ~/shared/dumps_1 features.csv
[*] Found 8 dump(s). Starting extraction...

[1/8] memdump_20260505_122448_clean.raw
      running linux.pslist...
  
```

Annotations in the image:

- A red box around `~/shared/dumps_1` is labeled "input directory".

Figure 4: Volatility plugin execution and CSV output

label_rootkit	label_injection	label_backdoor	hidden_proc_count	hooked_syscall_count	hidden_module_count	total_sockets	high_port_listeners	has_injection
0	0	0	31	0	0	235	0	0
0	0	1	23	0	0	243	2	0
1	0	1	21	0	1	251	2	0
1	0	0	24	0	1	235	0	0
0	1	0	24	0	0	235	0	1
0	1	1	19	0	0	251	2	1
1	1	0	21	0	1	243	2	1
1	1	1	21	0	1	243	2	1

Figure 5: Extracted features

3. Model Training

Trains the multi-output Decision Tree classifier with LOO-CV and saves the fitted model to disk.

```
memsnap train -i ./assets/features.csv -o ./model.pkl
```

```
(venv) [karim@T460 forensic-lab]$
(venv) [karim@T460 forensic-lab]$
(venv) [karim@T460 forensic-lab]$
(venv) [karim@T460 forensic-lab]$ memsnap train -i ./assets/features.csv -o ./model.pkl
Per-label L00 accuracy:
  label_rootkit: 1.00
  label_injection: 1.00
  label_backdoor: 0.38
Exact match accuracy: 0.38
Model saved to ./model.pkl

Learned rules:
  label_rootkit: hidden_module_count <= 0.5
  label_injection: has_injection <= 0.5
  label_backdoor: high_port_listeners <= 1.0
(venv) [karim@T460 forensic-lab]$
```

Figure 6: LOO-CV results and model serialisation

4. CLI Detection

Given a single dump, extracts features on-the-fly and outputs a per-label verdict.

```
memsnap detect \
  -m ./assets/model.pkl \
  ./dumps_1/memdump_20260505_122730_rootkit_backdoor.raw
```

```
(venv) [karim@T460 forensic-lab]$ memsnap detect -m ./assets/model.pkl ./dumps_1/memdump_20260505_122730_rootkit_backdoor.raw
Extracting features...
Rootkit : detected
Injection : not detected
Backdoor : detected
[ble: elapsed 325.663s (CPU 83.9%)] memsnap detect -m ./assets/model.pkl ./dumps_1/memdump_20260505_122730_rootkit_backdoor.raw
(venv) [karim@T460 forensic-lab]$
```

Figure 7: Single-dump verdict

5. Web Interface

The web application allows users to upload a memory dump and an optional custom model. Real-time progress is streamed via Server-Sent Events.

memsnap web

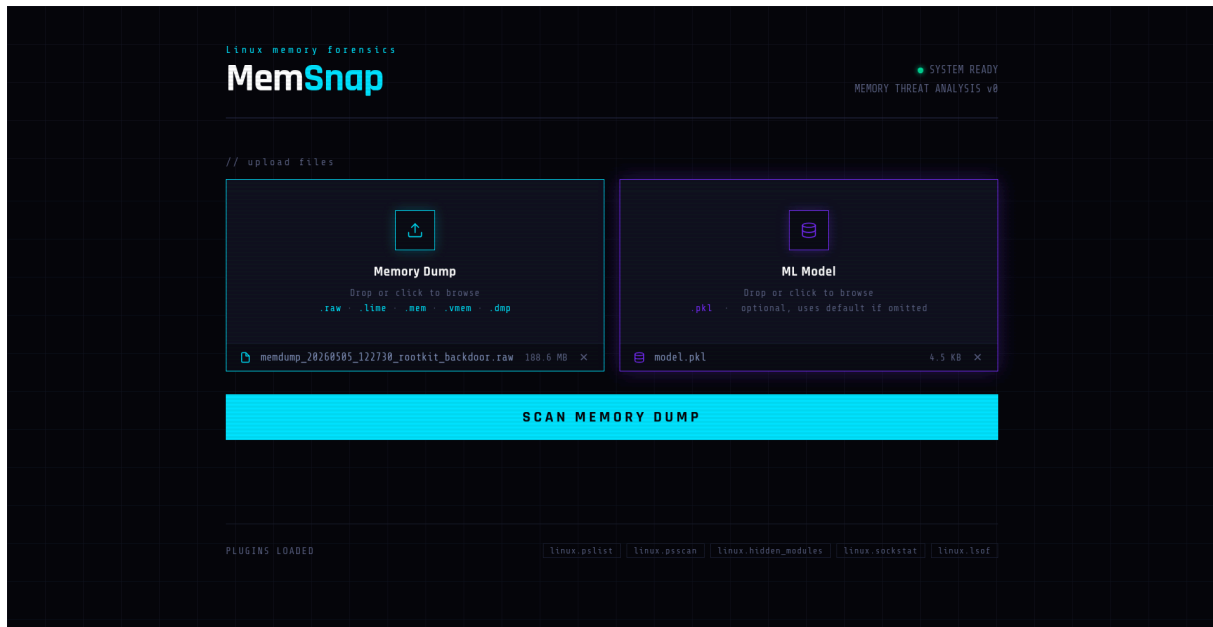


Figure 8: Upload the dump and model

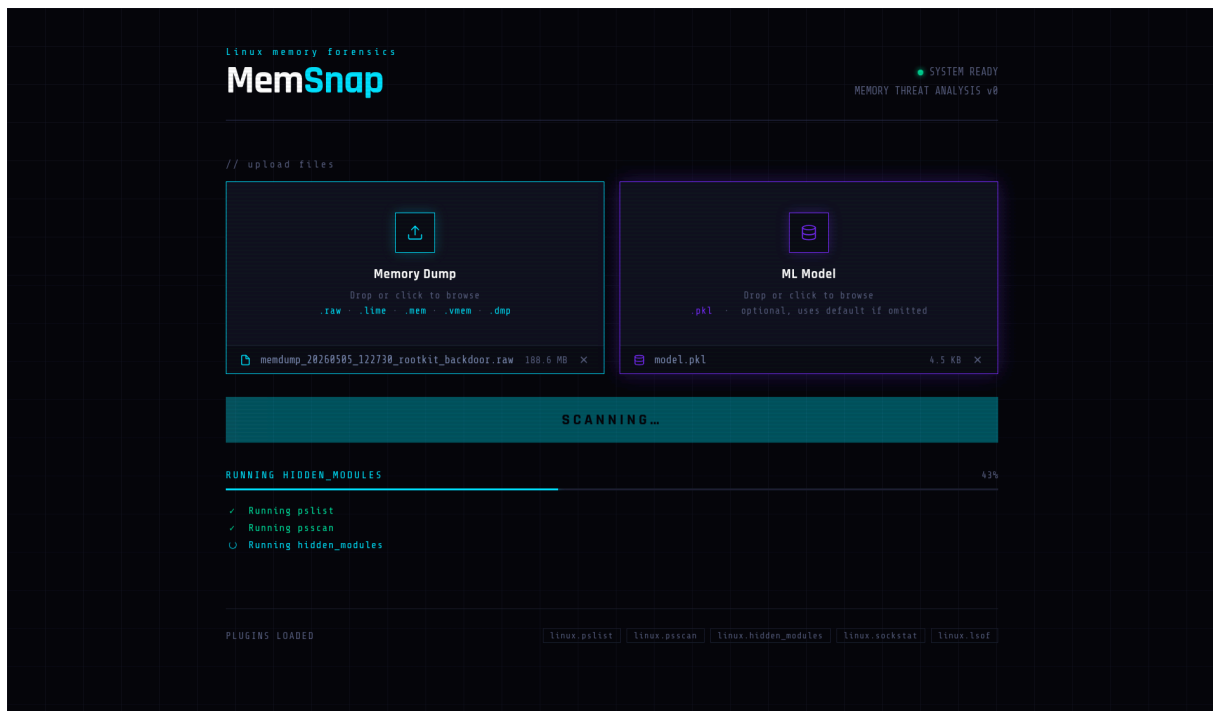


Figure 9: Extraction features

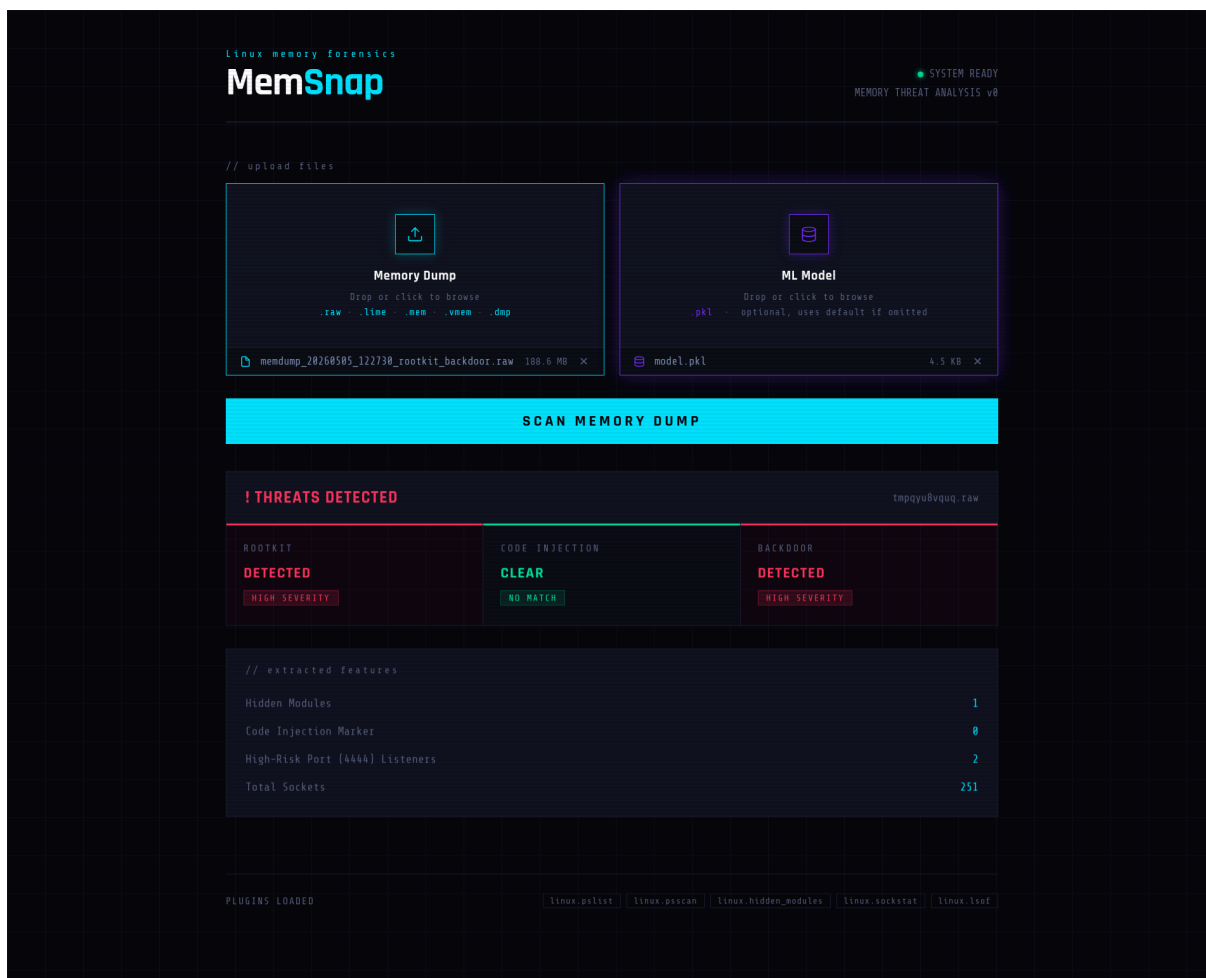


Figure 10: Threat verdict

The interface also displays the extracted feature values, giving the analyst insight into the detection rationale.

VIII. CONCLUSION

MemSnap successfully demonstrates a fully automated pipeline for Linux memory forensic analysis with AI-driven threat detection. The key contributions are:

- **Reproducible malware deployment** using pre-built assets.
- **Robust injection detection** via a creative use of an open file descriptor marker and Volatility's LSOF plugin.
- **A clean, user-friendly CLI and web interface** suitable for both technical and non-technical users.

While the proof-of-concept is limited by a small dataset and imperfect backdoor detection, it provides a solid foundation for further research and development.

IX. FUTURE WORK

- Add a backdoor-specific feature (process command-line analysis) to resolve the current misclassification.
- Expand the dataset with public memory samples and synthetic constrained-augmentation.
- Improve extraction performance through plugin parallelisation.
- Generate a formal PDF investigation report for each analysed dump.
- Explore more sophisticated models (Random Forest, XGBoost) once the dataset grows.
- Package the full tool (including the VM setup) into a single virtual appliance for educational use.